

THE FORWARD DEPLOYMENT CHALLENGE

Synq AI Engineering Event | 29 August 2026 | Duration: 8 hours

What this is actually testing

A forward deployed engineer walks into a client's environment, not a clean sandbox. The APIs are undocumented and flaky. The data disagrees with itself. The one person who understands operations is about to retire. And the client does not want a demo; they want a system that runs unattended on Monday morning, takes real actions, and does not do anything twice, wrongly, or silently.

Anyone can make an LLM answer questions. Almost nobody builds automations that survive contact with production. Today you prove which one you are. This challenge is deliberately too large to complete in eight hours. You are being evaluated on what you choose to build, how it behaves when things go wrong, and whether you can defend every decision your system makes.

The scenario

Meridian Freight Private Limited (fictional) runs around 400 trucks across North India. You have been deployed into their environment to ship one production automation: breakdown-to-resolution. When a vehicle breaks down mid-trip, today a human scrambles across five systems for forty minutes. Meridian wants it handled in seconds, correctly, with a human approving only the irreversible steps.

You have two kinds of material: their live systems, and their tribal knowledge. Both are messy on purpose, because that is what real deployments look like.

The environment you are deploying into

The breakdown queue (your trigger)

Meridian's breakdown tickets are provided as a queue file, tickets.json. Your pipeline processes this queue end to end, unattended. The queue contains, by design: duplicate ticket ids from a sync fault, and records with missing or broken critical fields. Duplicates are processed exactly once; broken records are quarantined with an alert, never silently dropped and never allowed to crash the run. Your actions (work orders, client messages) are written to a standard outputs format defined in the docs; a duplicate work order or duplicate client message for the same ticket is a scored failure, exactly as if each write cost real money. One more rule that replaces a thousand words: running your entire pipeline twice, back to back, must produce identical outputs. Nothing doubled, nothing lost.

Fair warning, and read this twice: in the final hour you receive a second, smaller ticket file. It will not look exactly like the main queue, because in the real world the client's IT team changes formats without telling you. How your system handles that file, live, is scored.

Static corpus (provided as files)

- Maintenance log with free-text mechanic notes, some in mixed Hindi-English
- Client email threads, some of which contradict the structured data

- A 14-year veteran dispatcher's interview transcript containing the real operating rules: seasonal route restrictions by vehicle type, clients whose contractual SLA and actual expectations differ, exceptions that override the obvious choice
- Driver roster and breakdown ticket history

Personal data (phone numbers, ID numbers, license numbers) is planted throughout. If any raw personal datum ever appears in an outbound action, a log line visible to evaluators, or an API response you serve, your score is capped at 50 of 100. This gate mirrors the real product and is not negotiable.

What you must ship

Part A: The context foundation

Ingest the live APIs and the static corpus into a unified, entity-resolved store with personal data masked at ingestion. The same vehicle appears in multiple formats; the same client under multiple names; facts change over time and sources conflict. Where they conflict, your system applies a documented precedence rule, never a silent guess. Expose one query interface that returns answers with citations to source records, and says plainly when the data is insufficient. Confident unsupported answers carry negative marks.

Part B: The automation (the core of this challenge)

Build the breakdown-to-resolution pipeline, running unattended, consuming the ticket queue:

- **Step 1:** Validate each ticket: duplicates processed exactly once, records with missing critical fields quarantined with an alert, never a crash
- **Step 2:** Enrich with full context: which vehicle, which driver, which trip, which client, which SLA, current location on route, maintenance history relevant to the failure
- **Step 3:** Classify severity and required action using the dispatcher's encoded rules, not vibes: the transcript's rules must be structured, queryable logic that the pipeline consults and cites
- **Step 4:** Select a replacement vehicle that is genuinely eligible: available, route-permitted for the season per the expert rules, not overdue on maintenance, and not already assigned
- **Step 5:** Record exactly one work order in your outbox, no matter how many times the ticket appears in the queue
- **Step 6:** Draft the client notification and queue it behind a human approval gate with the full context and citations shown to the approver; on approval, record it as sent in your outbox exactly once
- **Step 7:** Write an audit record for every step: what was decided, on what data, under which rule, by whom or by what

Production requirements (this is where it gets hard)

- **Idempotency:** Duplicate queue records and full pipeline re-runs must never produce duplicate work orders or duplicate client messages
- **Re-runability:** Your pipeline run twice back to back produces identical outputs; state survives between runs

- **Change tolerance:** When the surprise file's changed format hits, your system must degrade safely and raise an alert rather than corrupt data or crash. Fast, clean adaptation is scored
- **Observability:** Structured logs or a minimal dashboard from which an evaluator can reconstruct what your system did and why, for any event, in under a minute
- **Deployability:** Your entire system must start on a clean machine with a single documented command. If we cannot run it, it does not exist

The final hour

In the final hour, in front of the evaluators: you receive the surprise ticket file and process it live; you run your full pipeline twice back to back to prove nothing doubles and nothing is lost; and you answer live questions your system has never seen. Systems built for the happy path fail here in the first ten minutes, and that failure is the single most informative signal of this entire day.

Scoring (100 points)

- **35 pts:** Automation correctness: exactly-once actions across duplicates and full re-runs, clean quarantine of broken records, safe handling of the surprise file
- **25 pts:** Context quality: entity resolution against ground truth, temporal and conflict handling, grounded answers on a hidden question set with negative marking for hallucination
- **15 pts:** Expert-rule encoding: decisions that respect and cite the dispatcher's rules, including at least one case where the rule overrides the obvious choice
- **15 pts:** Production hygiene: audit trail, observability, security, one-command deploy
- **10 pts:** Defense: a fifteen-minute walkthrough in which you justify architecture choices, trade-offs, and what you deliberately cut. Wrong-but-reasoned beats accidental-and-right

Hard gate: any raw personal data in any outbound action, served response, or evaluator-visible log caps the total at 50.

Rules and logistics

- Individual work. Everything demoed must be built during the event window.
- Any stack, any LLM, internet allowed, AI assistance allowed. What is not allowed is shipping behavior you cannot explain line by line.
- Checkpoints: hour 3, ingestion and entity resolution report; hour 5, pipeline processing the full queue end to end; hour 7, the final hour begins.
- The environment's misbehavior is uniform for all candidates and controlled by the evaluation harness, not by mood.

A closing note

Nobody finishes this cleanly in eight hours. It is sized so that your prioritisation is visible: what you hardened, what you stubbed, what you refused to fake, and whether the thing you shipped tells the truth under attack. That is the job. Ship something that survives.